yulul-hypothesis-testing

September 15, 2024

About Yulu

Yulu is India's leading micro-mobility service provider, which offers unique vehicles for the daily commute. Starting off as a mission to eliminate traffic congestion in India, Yulu provides the safest commute solution through a user-friendly mobile app to enable shared, solo and sustainable commuting.

Yulu zones are located at all the appropriate locations (including metro stations, bus stands, office spaces, residential areas, corporate offices, etc) to make those first and last miles smooth, affordable, and convenient!

Yulu has recently suffered considerable dips in its revenues. They have contracted a consulting company to understand the factors on which the demand for these shared electric cycles depends. Specifically, they want to understand the factors affecting the demand for these shared electric cycles in the Indian market.

How you can help here?

The company wants to know:

Which variables are significant in predicting the demand for shared electric cycles in the Indian market?

How well those variables describe the electric cycle demands

[35]:
```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
import seaborn as sns
```

[36]:
```
# Importing data
df = pd.read_csv('Yulu_Hypothesis_test.csv')
```

Data Exploration

[37]:
```
# no of rows amd columns in dataset
print(f"# rows: {df.shape[0]} \n# columns: {df.shape[1]}")
```

```
# rows: 10886
# columns: 12
```

[38]:
```
df.head()
```

[38]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2011-01-01 00:00:00	1	0	0	1	9.84	14.395	81	0.0	3	13	16
1	2011-01-01 01:00:00	1	0	0	1	9.02	13.635	80	0.0	8	32	40
2	2011-01-01 02:00:00	1	0	0	1	9.02	13.635	80	0.0	5	27	32
3	2011-01-01 03:00:00	1	0	0	1	9.84	14.395	75	0.0	3	10	13
4	2011-01-01 04:00:00	1	0	0	1	9.84	14.395	75	0.0	0	1	1

[39]:
```
df.tail()
```

[39]:

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
10881	2012-12-19 19:00:00	4	0	1	1	15.58	19.695	50	26.0027	7	329	336
10882	2012-12-19 20:00:00	4	0	1	1	14.76	17.425	57	15.0013	10	231	241
10883	2012-12-19 21:00:00	4	0	1	1	13.94	15.910	61	15.0013	4	164	168
10884	2012-12-19 22:00:00	4	0	1	1	13.94	17.425	61	6.0032	12	117	129
10885	2012-12-19 23:00:00	4	0	1	1	13.12	16.665	66	8.9981	4	84	88

[40]:
```
print(df.info())
print()
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   datetime    10886 non-null  object
 1   season      10886 non-null  int64
 2   holiday     10886 non-null  int64
 3   workingday  10886 non-null  int64
 4   weather     10886 non-null  int64
 5   temp        10886 non-null  float64
```

```
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64
 10  registered  10886 non-null  int64
 11  count       10886 non-null  int64
dtypes: float64(3), int64(8), object(1)
memory usage: 1020.7+ KB
None
```

	season	holiday	workingday	weather	temp
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.00000
mean	2.506614	0.028569	0.680875	1.418427	20.23086
std	1.116174	0.166599	0.466159	0.633839	7.79159
min	1.000000	0.000000	0.000000	1.000000	0.82000
25%	2.000000	0.000000	0.000000	1.000000	13.94000
50%	3.000000	0.000000	1.000000	1.000000	20.50000
75%	4.000000	0.000000	1.000000	2.000000	26.24000
max	4.000000	1.000000	1.000000	4.000000	41.00000

	atemp	humidity	windspeed	casual	registered
count	10886.000000	10886.000000	10886.000000	10886.000000	10886.000000
mean	23.655084	61.886460	12.799395	36.021955	155.552177
std	8.474601	19.245033	8.164537	49.960477	151.039033
min	0.760000	0.000000	0.000000	0.000000	0.000000
25%	16.665000	47.000000	7.001500	4.000000	36.000000
50%	24.240000	62.000000	12.998000	17.000000	118.000000
75%	31.060000	77.000000	16.997900	49.000000	222.000000
max	45.455000	100.000000	56.996900	367.000000	886.000000

	count
count	10886.000000
mean	191.574132
std	181.144454
min	1.000000
25%	42.000000
50%	145.000000
75%	284.000000
max	977.000000

[41]:
```
# Converting datatime column from object type to datetime type
df['datetime'] = pd.to_datetime(df['datetime'])

cat_cols= ['season', 'holiday', 'workingday', 'weather']
for col in cat_cols:
    df[col] = df[col].astype('object')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10886 entries, 0 to 10885
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   datetime    10886 non-null  datetime64[ns]
 1   season      10886 non-null  object
 2   holiday     10886 non-null  object
 3   workingday  10886 non-null  object
 4   weather     10886 non-null  object
 5   temp        10886 non-null  float64
 6   atemp       10886 non-null  float64
 7   humidity    10886 non-null  int64
 8   windspeed   10886 non-null  float64
 9   casual      10886 non-null  int64
 10  registered  10886 non-null  int64
 11  count       10886 non-null  int64
dtypes: datetime64[ns](1), float64(3), int64(4), object(4)
memory usage: 1020.7+ KB
```

[41]:

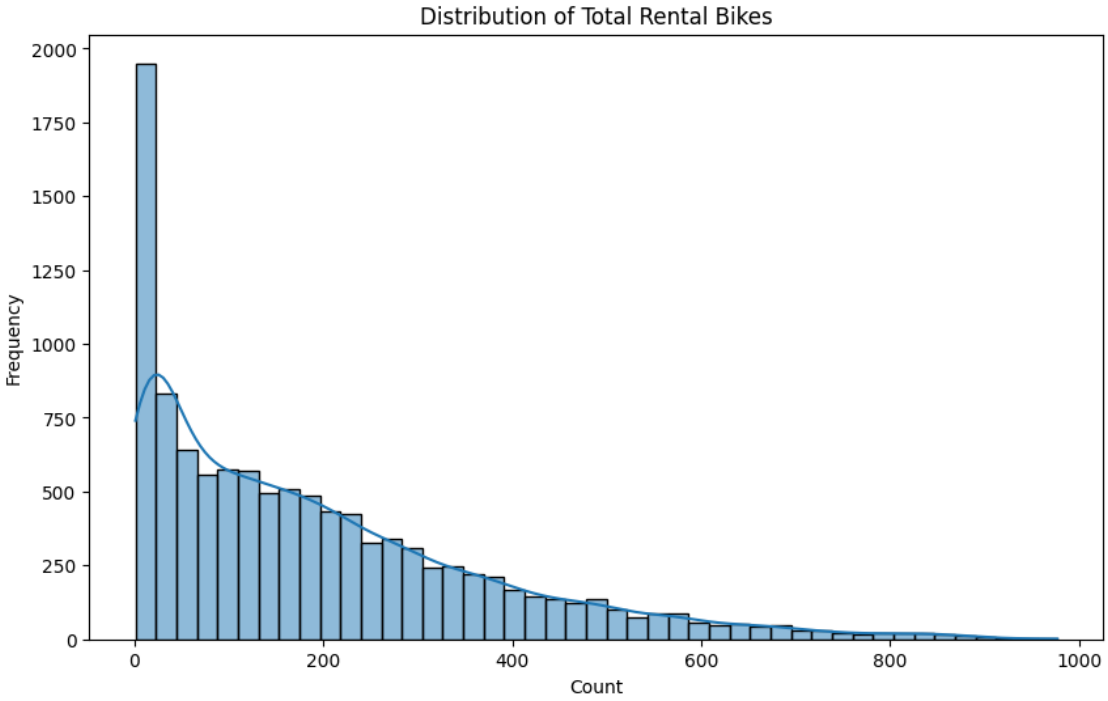
[42]:
```
# Check for missing values
print(df.isnull().sum())
```

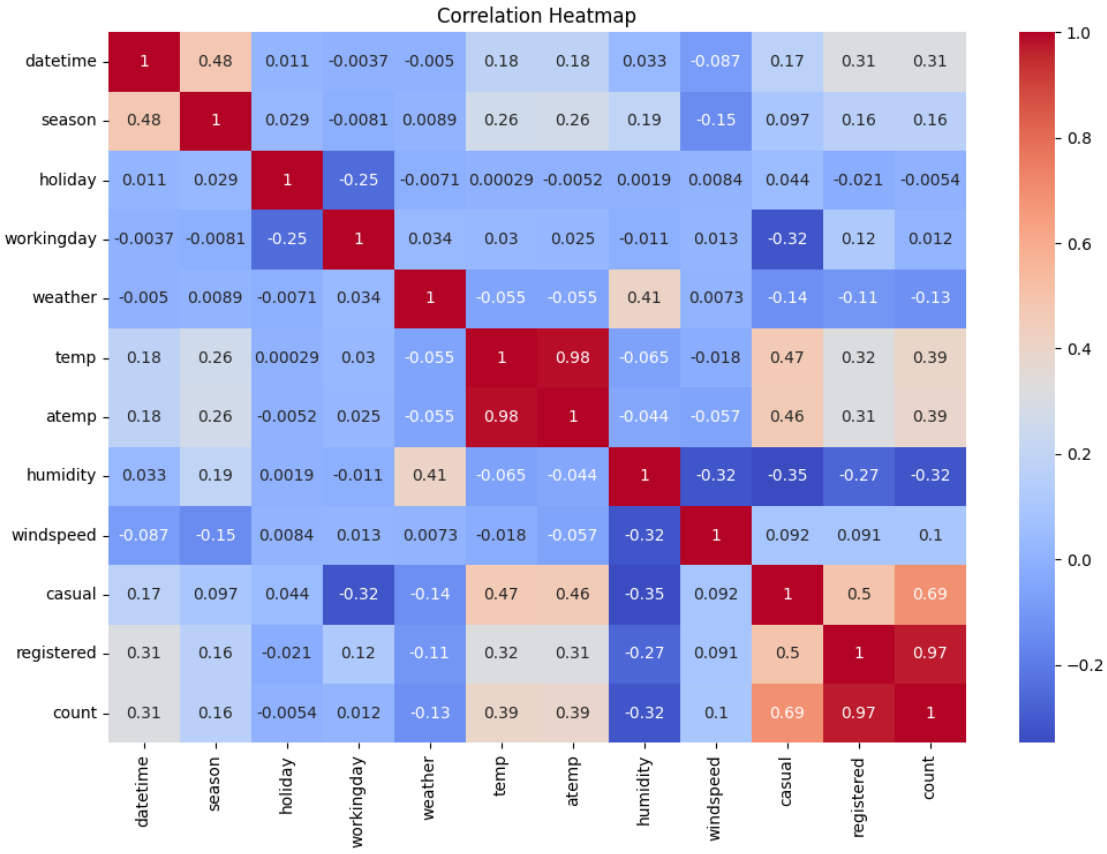
```
datetime      0
season        0
holiday       0
workingday    0
weather       0
temp          0
atemp         0
humidity      0
windspeed     0
casual        0
registered    0
count         0
dtype: int64
```

Data Visualization

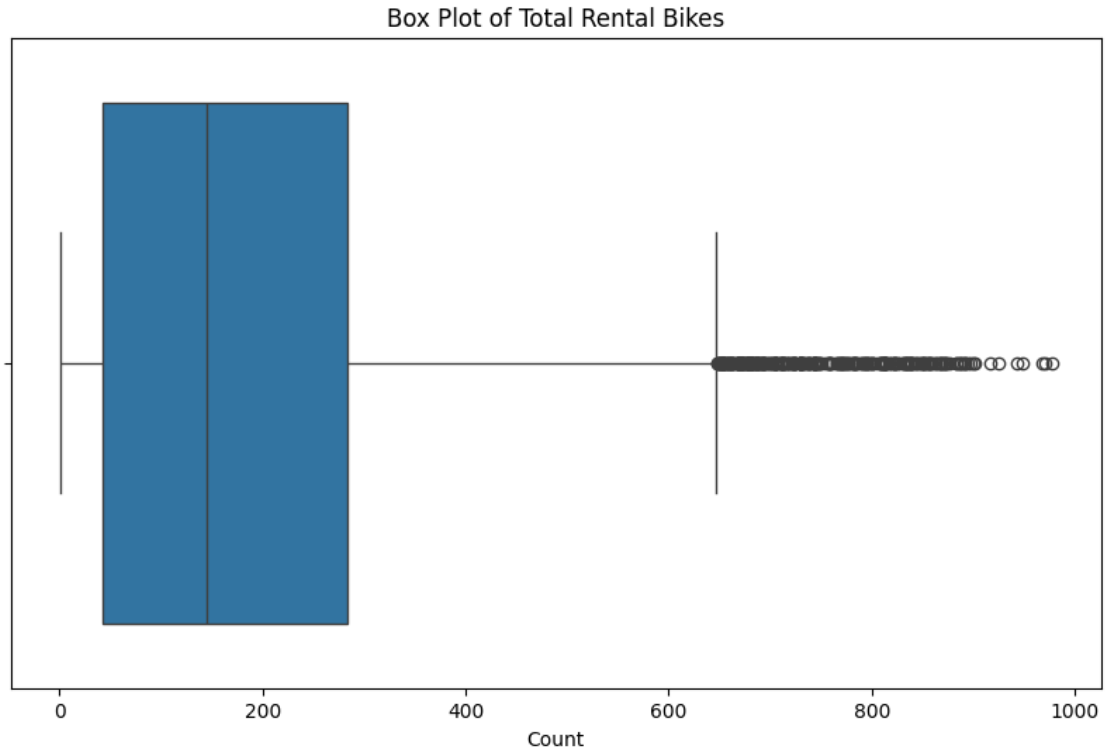
[43]:
```
# Distribution of total rental bikes
plt.figure(figsize=(10, 6))
sns.histplot(df['count'], kde=True)
plt.title('Distribution of Total Rental Bikes')
plt.xlabel('Count')
plt.ylabel('Frequency')
plt.show()
```

```
# Correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

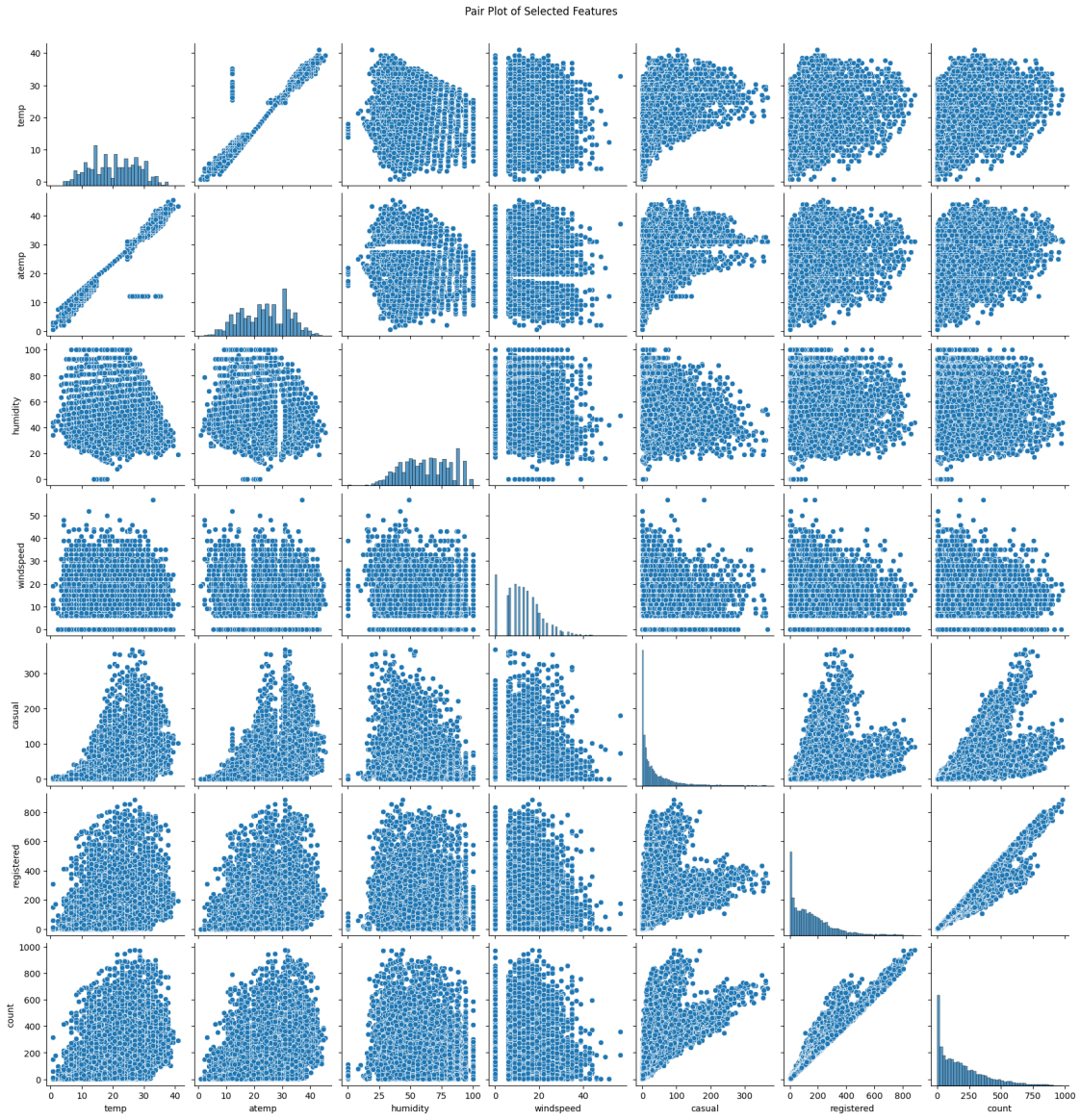




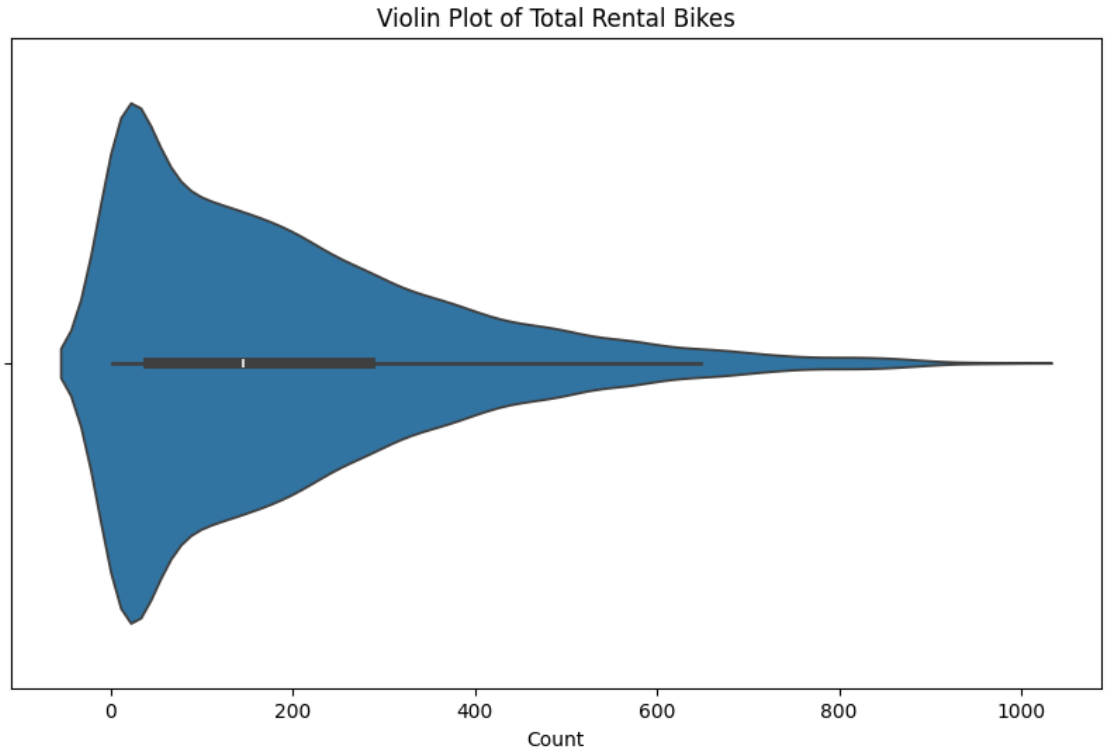
[44]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['count'])
plt.title('Box Plot of Total Rental Bikes')
plt.xlabel('Count')
plt.show()
```



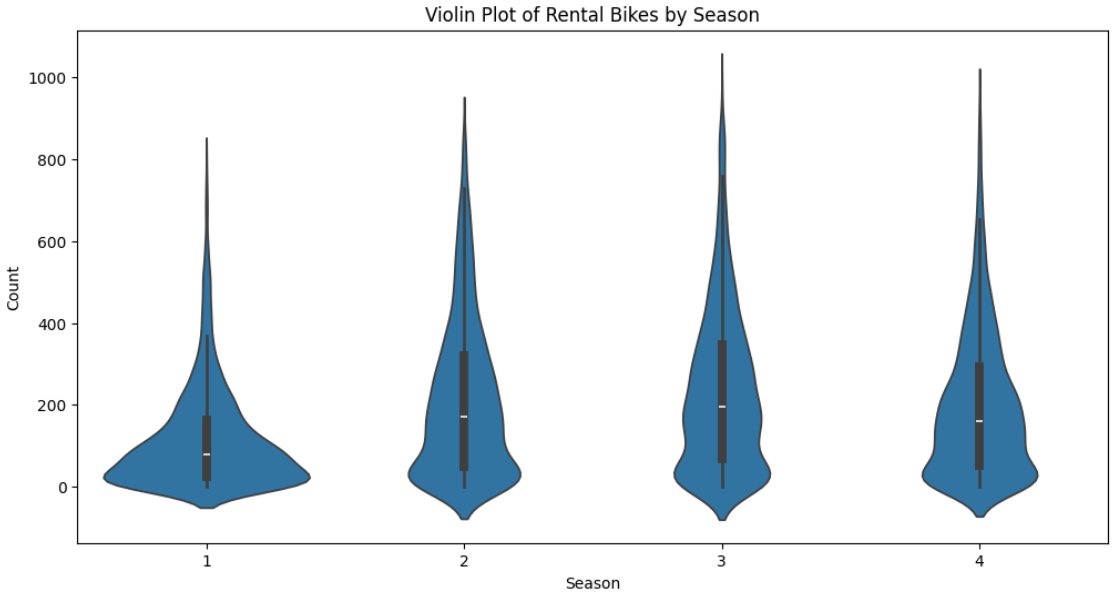
[45]:
```
# Pair Plot for Correlations
features = ['temp', 'atemp', 'humidity', 'windspeed', 'casual', 'registered',␣
 ↪'count']
sns.pairplot(df[features])
plt.suptitle('Pair Plot of Selected Features', y=1.02)
plt.show()
```



[46]:
```
plt.figure(figsize=(10, 6))
sns.violinplot(x=df['count'])
plt.title('Violin Plot of Total Rental Bikes')
plt.xlabel('Count')
plt.show()
```



[47]:
```
# Violin Plot by Season
plt.figure(figsize=(12, 6))
sns.violinplot(x='season', y='count', data=df)
plt.title('Violin Plot of Rental Bikes by Season')
plt.xlabel('Season')
plt.ylabel('Count')
plt.show()
```



Violine Plot :

A violin plot is a combination of a box plot and a kernel density estimation plot. It provides a more detailed view of the distribution of a variable across different categories.

Distribution Shapes: The violin plots reveal the overall shape of the distributions for each season. They are generally skewed to the right, indicating that there are fewer days with very high rental counts compared to days with lower counts.

Median Values: The white dot within each violin represents the median rental count for that season.

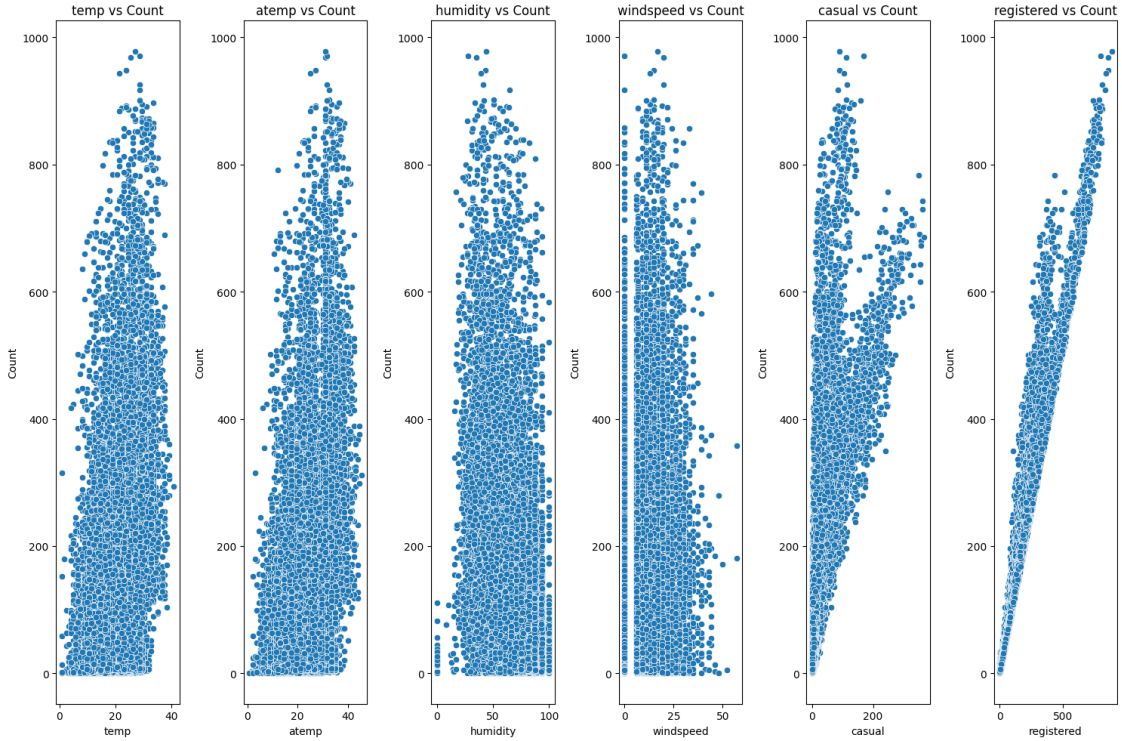
Interquartile Range (IQR): The box within each violin shows the IQR, which represents the middle 50% of the data.

Outliers: The individual points outside the whiskers indicate potential outliers, which are data points that are significantly different from the main body of the data.

Seasonal Variations: The violin plots suggest that the distribution of rental counts varies across seasons. Some seasons have a higher overall level of rentals, while others have a lower level.

[48]:
```
# Correlation Scatter Plots

features = ['temp', 'atemp', 'humidity', 'windspeed', 'casual', 'registered']

plt.figure(figsize=(15, 10))
for i, feature in enumerate(features):
    plt.subplot(1, len(features), i + 1)
    sns.scatterplot(x=df[feature], y=df['count'])
    plt.title(f'{feature} vs Count')
    plt.xlabel(feature)
```

```
    plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



Insights:

Seasonal Peaks: Season 3 (fall) appears to have the highest median rental count, suggesting that it might be the peak season for bike rentals.

Distribution Variability: The IQR and the shape of the violins indicate that the variability of rental counts differs across seasons. Some seasons have a wider range of rental counts, while others have a more concentrated distribution.

Outliers: The presence of outliers suggests that there are occasional days with exceptionally high or low rental counts, which could be due to factors like weather events, holidays, or special events.

Facet Grid for Distribution by Weather

It's is a type of visualization that allows you to compare distributions across different categories or groups. In this case, the facet grid is showing the distribution of total bike rentals for each weather condition.

Weather Impact:

The facet grid confirms that weather conditions significantly influence bike rental demand. The distributions is highest in Spring following Summer, Rain and finally in Winter the business is

considerably low

Peak Rental Activity:

The peak locations of the distributions provide insights into the typical rental levels for each weather condition.

Variability Analysis:

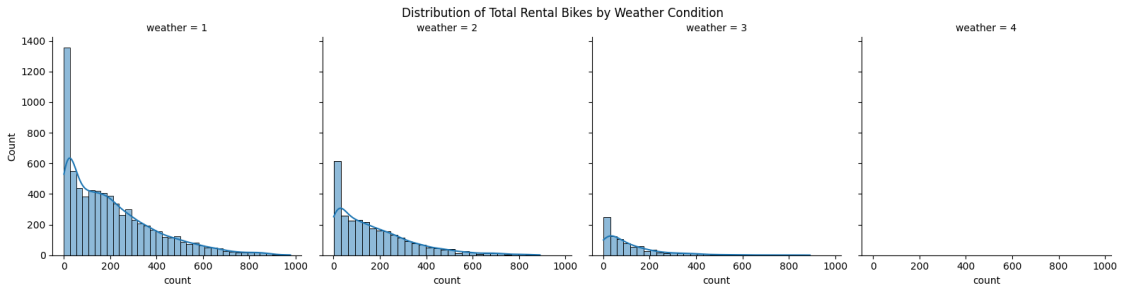
The varying spread of the distributions suggests that the impact of weather on rental counts is not uniform. Winter and Summer conditions lead to more consistent rental patterns than others.

[49]:
```
unique_weather_values = df['season'].unique()
print("Unique values in the 'weather' column:")
print(unique_weather_values)

# season (1: spring, 2: summer, 3: fall, 4: winter)
```

```
Unique values in the 'weather' column:
[1 2 3 4]
```

[50]:
```
# Facet Grid for Distribution by Weather

plt.figure(figsize=(12, 8))
sns.FacetGrid(df, col='weather', col_wrap=4, height=4).map(sns.histplot,␣
 ↪'count', kde=True)
plt.suptitle('Distribution of Total Rental Bikes by Weather Condition', y=1.02)
plt.show()
```

```
<Figure size 1200x800 with 0 Axes>
```



[50]:

Correlation Heatmap

This is a powerful visualization that shows the strength and direction of relationships between numerical variables in your dataset

Here each cell represents the correlation coefficient between two features, ranging from -1 to 1:

Positive Correlation (1): As one feature increases, the other also increases.

Here casual and registered have a high positive correlation with count, it suggests that both

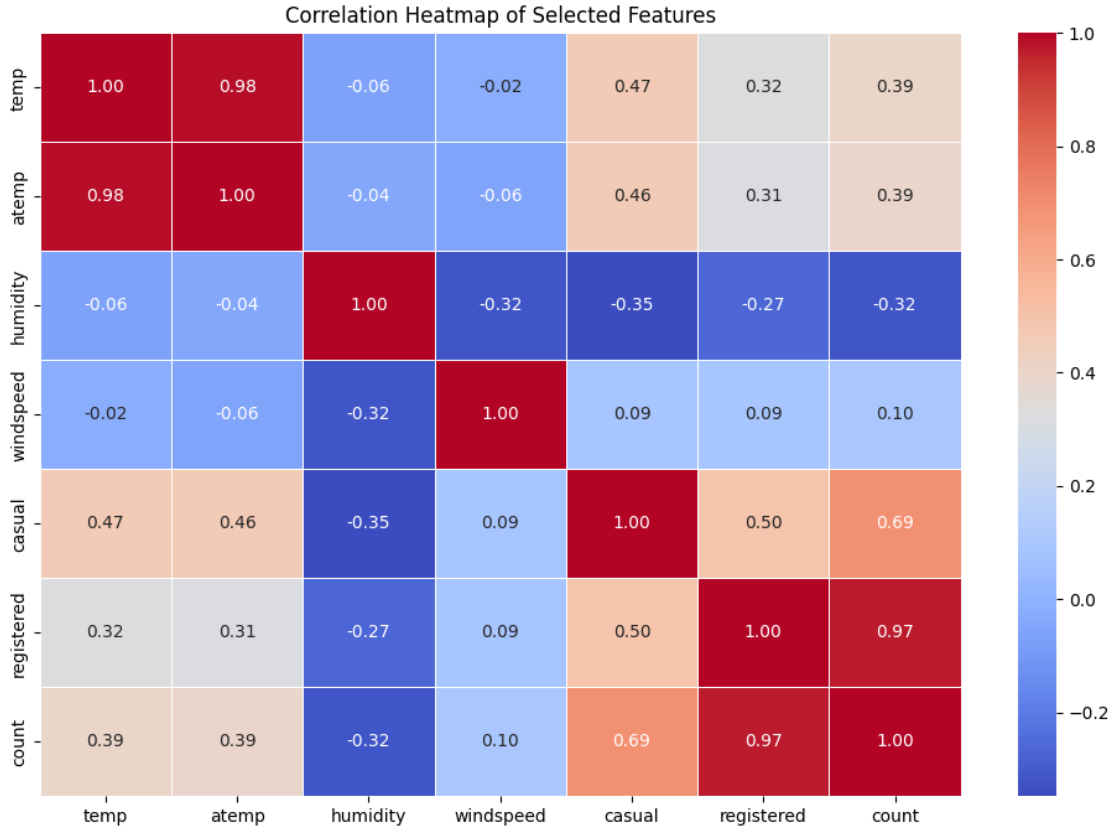
Negative Correlation (-1): As one feature increases, the other decreases.

Here windspeed and count have a high negative correlation, it indicates that higher wind spe

No Correlation (0): There's no linear relationship between the features.

And humidity and atemp have a low correlation with count, it might imply that changes in humi

[51]:
```
# Correlation Heatmap

plt.figure(figsize=(12, 8))
correlation_matrix = df[['temp', 'atemp', 'humidity', 'windspeed', 'casual',␣
 ↪'registered', 'count']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',␣
 ↪linewidths=0.5)
plt.title('Correlation Heatmap of Selected Features')
plt.show()
```



[52]:
```
# minimum datetime and maximum datetime
df['datetime'].min(), df['datetime'].max()
```

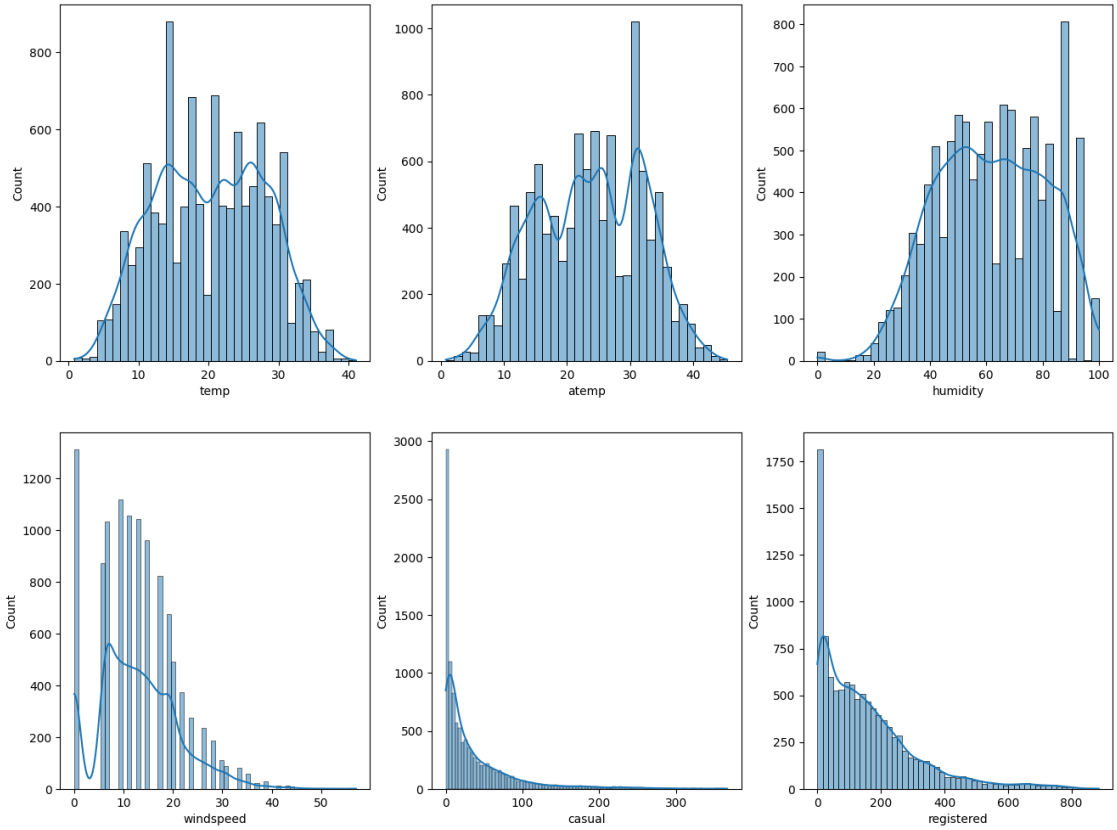
[52]:
```
(Timestamp('2011-01-01 00:00:00'), Timestamp('2012-12-19 23:00:00'))
```

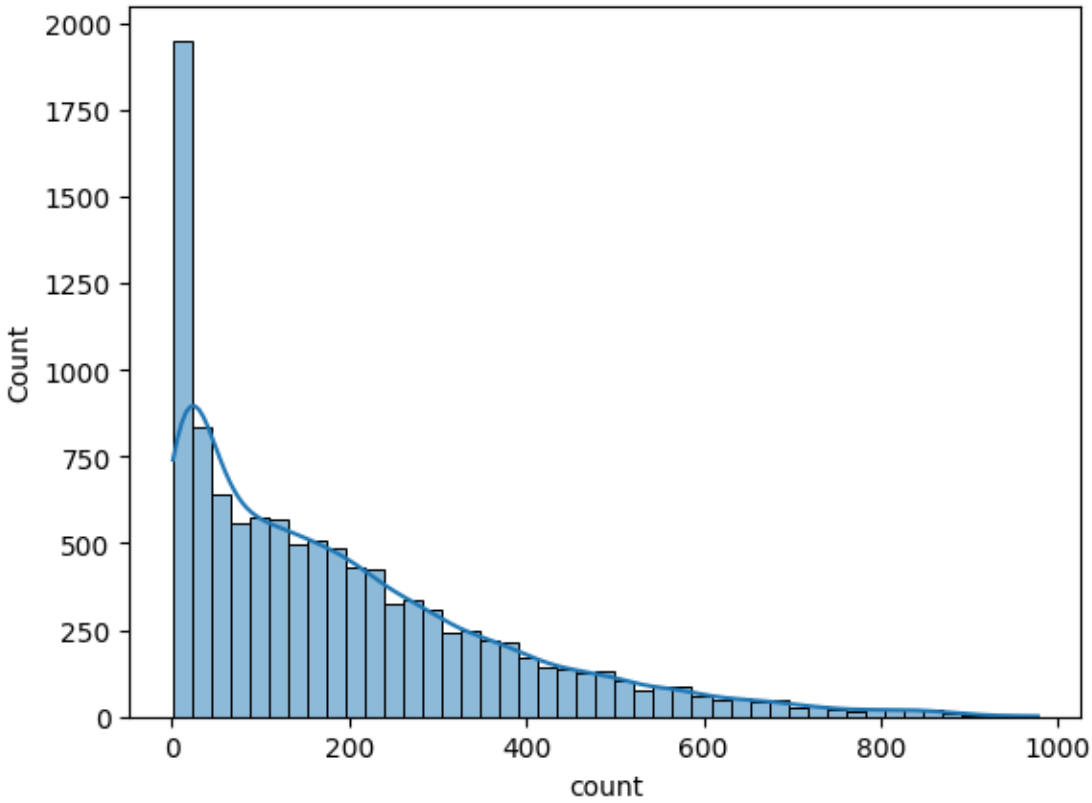
[53]:
```
# number of unique values in each categorical columns
df[cat_cols].melt().groupby(['variable', 'value'])[['value']].count()
```

[53]:

variable	value	value
holiday	0	10575
	1	311
season	1	2686
	2	2733
	3	2733
	4	2734
weather	1	7192
	2	2834
	3	859
	4	1
workingday	0	3474
	1	7412

[54]:
```
# understanding the distribution for numerical variables
num_cols = ['temp', 'atemp', 'humidity', 'windspeed', 'casual',␣
 ↪'registered','count']

fig, axis = plt.subplots(nrows=2, ncols=3, figsize=(16, 12))

index = 0
for row in range(2):
    for col in range(3):
        sns.histplot(df[num_cols[index]], ax=axis[row, col], kde=True)
        index += 1

plt.show()
sns.histplot(df[num_cols[-1]], kde=True)
plt.show()
```


Count
temp
atemp
humidity
windspeed
casual
registered



Casual, registered and count somewhat looks like Log Normal Distrinution

Temp, atemp and humidity looks like they follows the Normal Distribution

Windspeed follows the binomial distribution

Key Observations:

Skewness:

Temperature (temp) and Apparent Temperature (atemp): Both distributions are slightly right-skewed, indicating that there are more days with lower temperatures than higher ones.

Humidity: The distribution is moderately right-skewed, suggesting that there are more days with lower humidity levels.

Windspeed: The distribution is also slightly right-skewed, with a longer tail towards higher wind speeds.

Casual and Registered Users: Both distributions are heavily right-skewed, indicating that there are many days with low numbers of casual and registered users, but fewer days with high numbers.

Peak Locations:

Temperature (temp): The peak is around 20-25 degrees, suggesting that this temperature range is the most common.

Apparent Temperature (atemp): The peak is also around 20-25 degrees, reflecting a similar distribution to the actual temperature.

Humidity: The peak is around 50-60%, indicating that moderate humidity levels are most common.

Windspeed: The peak is around 10-15, suggesting that lower wind speeds are more frequent.

Casual and Registered Users: The peaks are around 0-50, indicating that most days have low numbers of casual and registered users.

Variability:

Temperature (temp) and Apparent Temperature (atemp): Both distributions have a moderate spread, suggesting that there is some variation in temperature and apparent temperature.

Humidity: The distribution has a wider spread, indicating more variation in humidity levels.

Windspeed: The distribution also has a wider spread, suggesting that wind speeds can vary significantly.

Casual and Registered Users: The distributions have a very wide spread, indicating that there can be large fluctuations in the number of casual and registered users from day to day.

Insights:

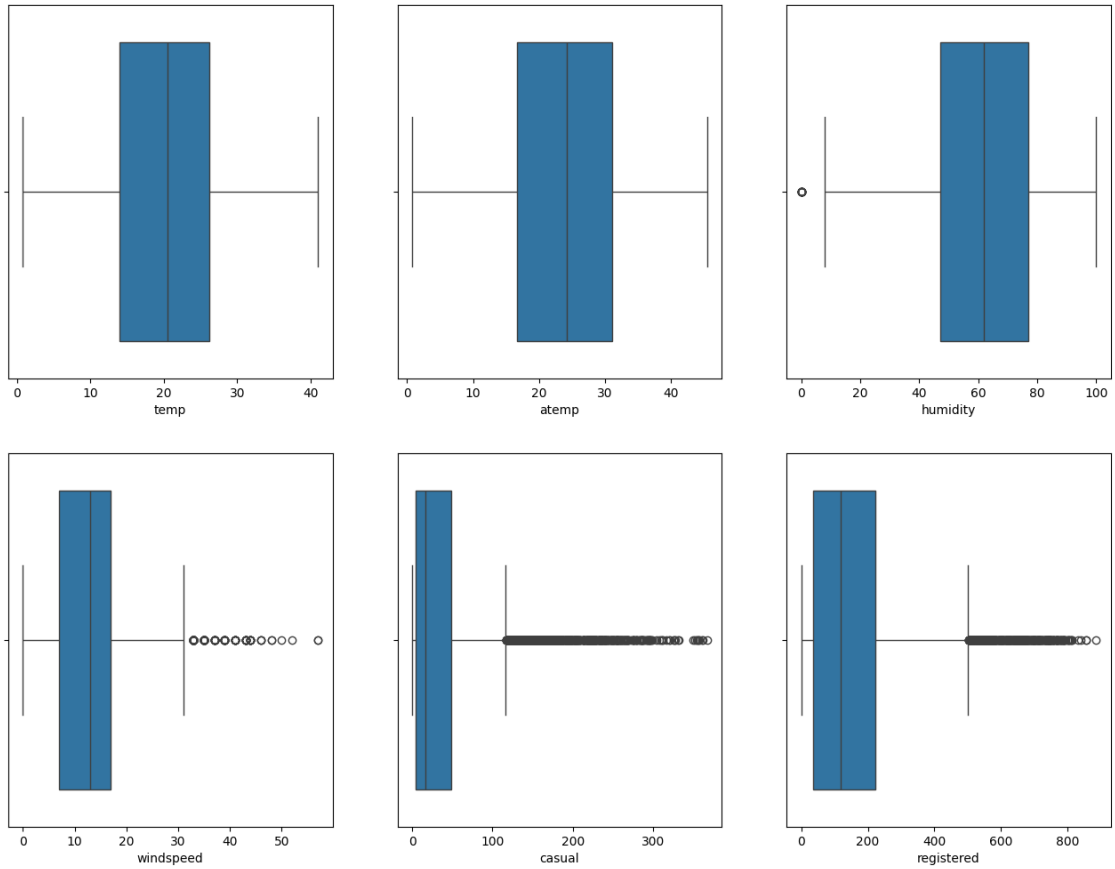
Temperature and Apparent Temperature: The distributions are similar, suggesting a strong correlation between the two variables.

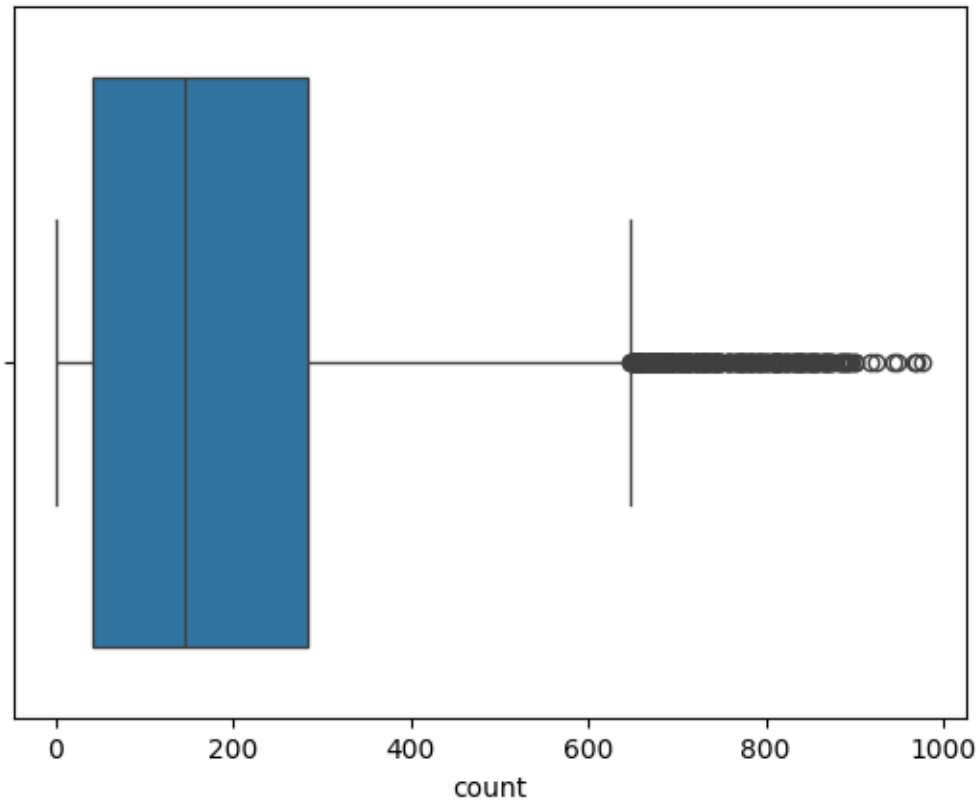
Humidity: The distribution is skewed towards lower humidity levels, indicating that dry conditions are more common.

Windspeed: There is a wider range of wind speeds, with lower speeds being more frequent.

User Behavior: The distributions for casual and registered users show that most days have low numbers of users, with occasional spikes in activity.

[55]:
```
fig, axis = plt.subplots(nrows=2, ncols=3, figsize=(16, 12))

index = 0
for row in range(2):
    for col in range(3):
        sns.boxplot(x=df[num_cols[index]], ax=axis[row, col])
        index += 1

plt.show()
sns.boxplot(x=df[num_cols[-1]])
plt.show()
```


0
10
20
30
40
temp
0
10
20
30
40
atemp
0
20
40
60
80
100
humidity
0
10
20
30
40
50
windspeed
0
100
200
300
casual
0
200
400
600
800
registered

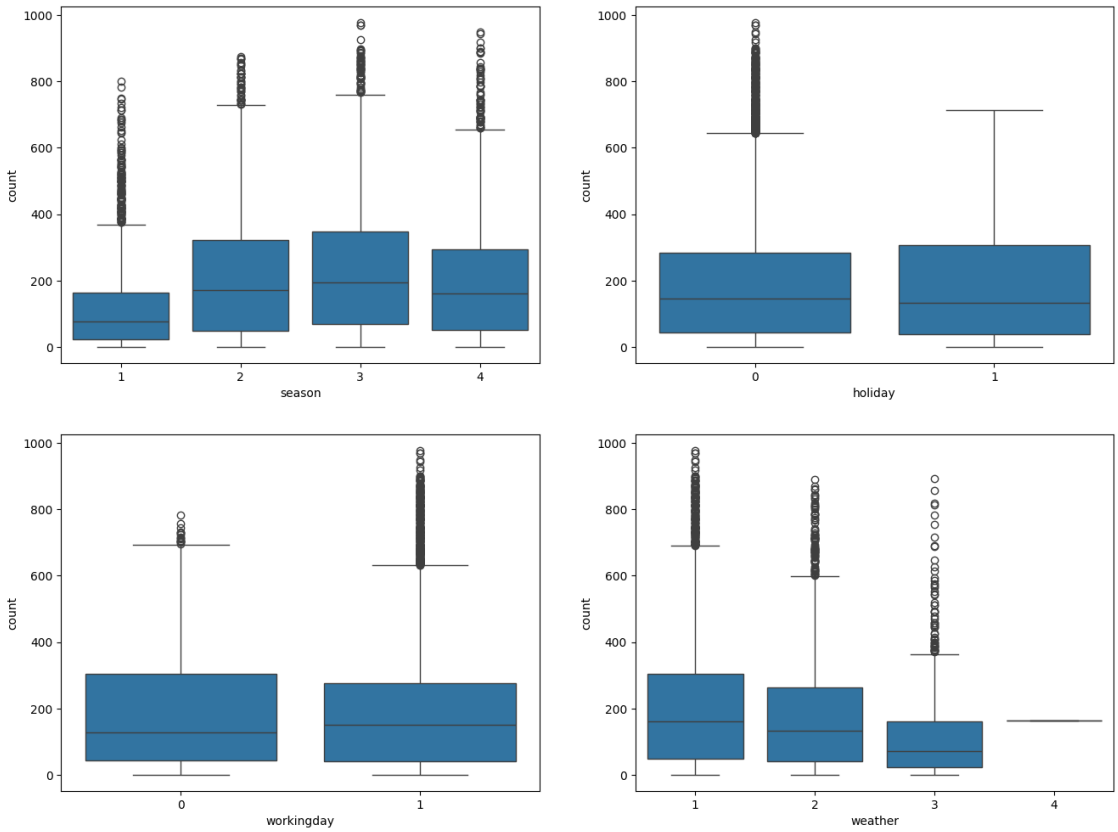


Looks like Count, Windspeed, casual and registered and count have outliers in the data

1 Bi-variate Analysis

plotting categorical variables against count using boxplots

[56]:
```
fig, axis = plt.subplots(nrows=2, ncols=2, figsize=(16, 12))

index = 0
for row in range(2):
    for col in range(2):
        sns.boxplot(data=df, x=cat_cols[index], y='count', ax=axis[row, col])
        index += 1

plt.show()
```



In Spring and Summer seasons more bikes are rented as compared to other seasons. Whenever its a holiday more bikes are rented.

It is also clear from the workingday also that whenever day is holiday or weekend, slightly more bikes were rented.

Whenever there is rain, thunderstorm, snow or fog, there were less bikes were rented.

[57]:
```
# understanding the correlation between count and numerical variables
df.corr()['count']
```

[57]:
```
datetime      0.310187
season        0.163439
holiday      -0.005393
workingday    0.011594
weather      -0.128655
temp          0.394454
atemp         0.389784
humidity     -0.317371
windspeed     0.101369
casual        0.690414
registered    0.970948
```

```
count         1.000000
Name: count, dtype: float64
```

2 Hypothesis Testing - 1

Null Hypothesis (H0): Weather is independent of the season

Alternate Hypothesis (H1): Weather is not independent of the season

Significance level (alpha): 0.05

[58]:
```
data_table = pd.crosstab(df['season'], df['weather'])
print("Observed values:")
data_table
```

```
Observed values:
```

[58]:

weather	1	2	3	4
season				
1	1759	715	211	1
2	1801	708	224	0
3	1930	604	199	0
4	1702	807	225	0

[60]:
```
val = stats.chi2_contingency(data_table)
expected_values = val[3]
expected_values

nrows, ncols = 4, 4
dof = (nrows-1)*(ncols-1)
print("degrees of freedom: ", dof)
alpha = 0.05


chi_sqr = sum([(o-e)**2/e for o, e in zip(data_table.values, expected_values)])
chi_sqr_statistic = chi_sqr[0] + chi_sqr[1]
print("chi-square test statistic: ", chi_sqr_statistic)

critical_val = stats.chi2.ppf(q=1-alpha, df=dof)
print(f"critical value: {critical_val}")

p_val = 1-stats.chi2.cdf(x=chi_sqr_statistic, df=dof)
print(f"p-value: {p_val}")

if p_val <= alpha:
    print("\nSince p-value is less than the alpha 0.05, We reject the Null␣
 ↪Hypothesis. Meaning that\
    Weather is dependent on the season.")
```

```
else:
    print("Since p-value is greater than the alpha 0.05, We do not reject the␣
 ↪Null Hypothesis")
```

```
degrees of freedom:  9
chi-square test statistic:  44.09441248632364
critical value: 16.918977604620448
p-value: 1.3560001579371317e-06

Since p-value is less than the alpha 0.05, We reject the Null Hypothesis.
Meaning that    Weather is dependent on the season.
```

Another Approach

[62]:
```
# Perform chi-square test
chi2_statistic, p_value, dof, expected_values = stats.
 ↪chi2_contingency(data_table)

print("Expected values:")
print(expected_values)

# Degrees of Freedom (dof)
print("Degrees of freedom: ", dof)

# Significance level (alpha)
alpha = 0.05

# Critical value
critical_val = stats.chi2.ppf(q=1-alpha, df=dof)
print(f"Critical value: {critical_val}")

# Print chi-square statistic
print(f"Chi-square test statistic: {chi2_statistic}")

# Print p-value
print(f"P-value: {p_value}")

# Decision based on p-value
if p_value <= alpha:
    print("\nSince the p-value is less than the alpha (0.05), we reject the␣
 ↪Null Hypothesis.")
    print("This indicates that the variables (e.g., weather and season) are␣
 ↪dependent.")
else:
    print("Since the p-value is greater than the alpha (0.05), we do not reject␣
 ↪the Null Hypothesis.")
```

```
    print("This indicates that there is not enough evidence to conclude that␣
 ↪the variables (e.g., weather and season) are dependent.")
```

```
Expected values:
[[1.77454639e+03 6.99258130e+02 2.11948742e+02 2.46738931e-01]
 [1.80559765e+03 7.11493845e+02 2.15657450e+02 2.51056403e-01]
 [1.80559765e+03 7.11493845e+02 2.15657450e+02 2.51056403e-01]
 [1.80625831e+03 7.11754180e+02 2.15736359e+02 2.51148264e-01]]
Degrees of freedom:  9
Critical value: 16.918977604620448
Chi-square test statistic: 49.158655596893624
P-value: 1.549925073686492e-07

Since the p-value is less than the alpha (0.05), we reject the Null Hypothesis.
This indicates that the variables (e.g., weather and season) are dependent.
```

3 Hypothesis Testing - 2

Null Hypothesis: Working day has no effect on the number of cycles being rented.

Alternate Hypothesis: Working day has effect on the number of cycles being rented.

Significance level (alpha): 0.05

We will use the 2-Sample T-Test to test the hypothess defined above

[67]:
```
# Group the data based on working day
data_group1 = df[df['workingday'] == 0]['count'].values  # Non-working days
data_group2 = df[df['workingday'] == 1]['count'].values  # Working days

# Calculate the variance for both groups
var_group1 = np.var(data_group1, ddof=1)  # Use ddof=1 for sample variance
var_group2 = np.var(data_group2, ddof=1)  # Use ddof=1 for sample variance

print(f"Variance of non-working days: {var_group1}")
print(f"Variance of working days: {var_group2}")

# Perform an independent t-test
t_statistic, p_value = stats.ttest_ind(a=data_group1, b=data_group2,␣
 ↪equal_var=True)

print(f"T-test statistic: {t_statistic}")
print(f"P-value: {p_value}")

# Significance level
alpha = 0.05

# Decision based on p-value
```

```
if p_value <= alpha:
    print("\nSince the p-value is less than the alpha (0.05), we reject the␣
 ↪Null Hypothesis.")
    print("This indicates that there is a significant difference in the mean␣
 ↪rental counts between working days and non-working days.")
else:
    print("Since the p-value is greater than the alpha (0.05), we do not reject␣
 ↪the Null Hypothesis.")
    print("This indicates that there is not enough evidence to conclude a␣
 ↪significant difference in the mean rental counts between working days and␣
 ↪non-working days.")
```

```
Variance of non-working days: 30180.03350064094
Variance of working days: 34045.29037312209
T-test statistic: -1.2096277376026694
P-value: 0.22644804226361348
Since the p-value is greater than the alpha (0.05), we do not reject the Null
Hypothesis.
This indicates that there is not enough evidence to conclude a significant
difference in the mean rental counts between working days and non-working days.
```

4 Welch's T-Test

Welch's t-test is a variation of the independent t-test that does not assume equal variances between the two groups. It is useful when the variances of the two groups are unequal.

[68]:
```
# Group the data based on working day
data_group1 = df[df['workingday'] == 0]['count'].values  # Non-working days
data_group2 = df[df['workingday'] == 1]['count'].values  # Working days

# Perform Welch's t-test
t_statistic, p_value = stats.ttest_ind(a=data_group1, b=data_group2,␣
 ↪equal_var=False)

print(f"Welch's t-test statistic: {t_statistic}")
print(f"P-value: {p_value}")

# Significance level
alpha = 0.05

# Decision based on p-value
if p_value <= alpha:
    print("\nSince the p-value is less than the alpha (0.05), we reject the␣
 ↪Null Hypothesis.")
    print("This indicates that there is a significant difference in the mean␣
 ↪rental counts between working days and non-working days.")
else:
```

```
    print("Since the p-value is greater than the alpha (0.05), we do not reject␣
 ↪the Null Hypothesis.")
    print("This indicates that there is not enough evidence to conclude a␣
 ↪significant difference in the mean rental counts between working days and␣
 ↪non-working days.")
```

```
Welch's t-test statistic: -1.2362580418223226
P-value: 0.21640312280695098
Since the p-value is greater than the alpha (0.05), we do not reject the Null
Hypothesis.
This indicates that there is not enough evidence to conclude a significant
difference in the mean rental counts between working days and non-working days.
```

5 Using Shapiro-wilk and levene test the rental counts are normally distributed for both working days and non-working days.

6 Hypothesis Testing - 2

Null Hypothesis: Working day has no effect on the number of cycles being rented.

Alternate Hypothesis: Working day has effect on the number of cycles being rented.

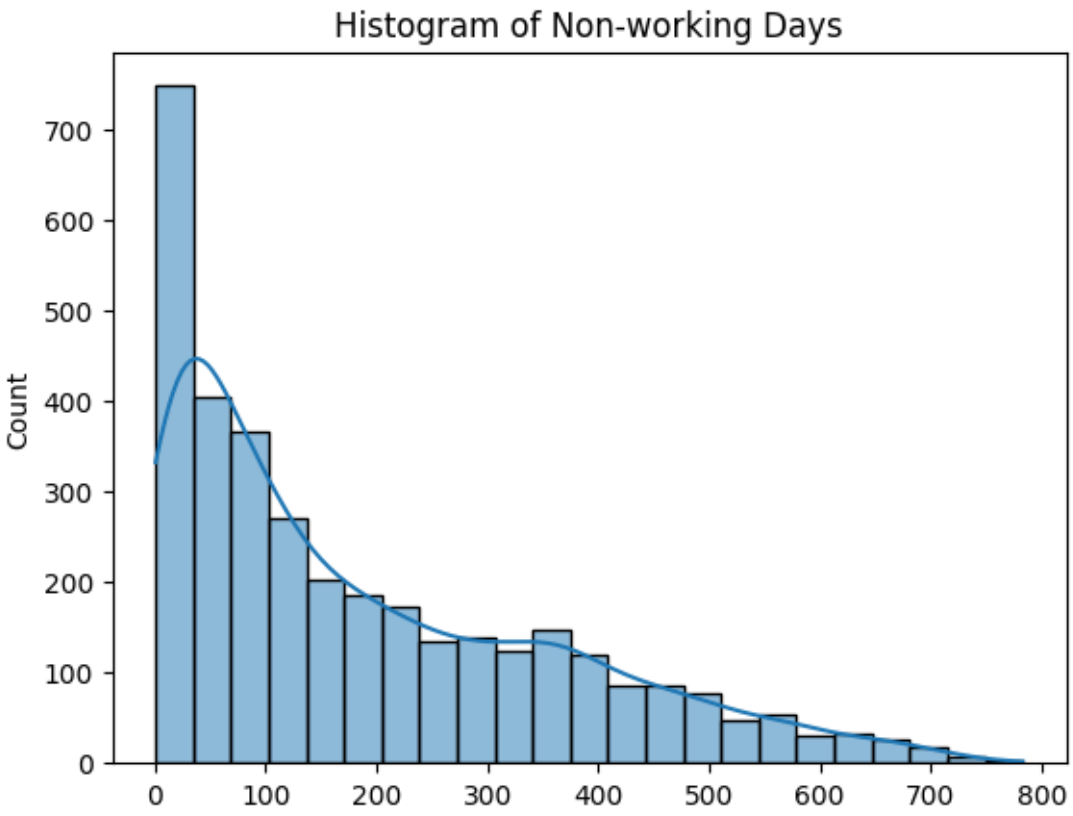
Significance level (alpha): 0.05

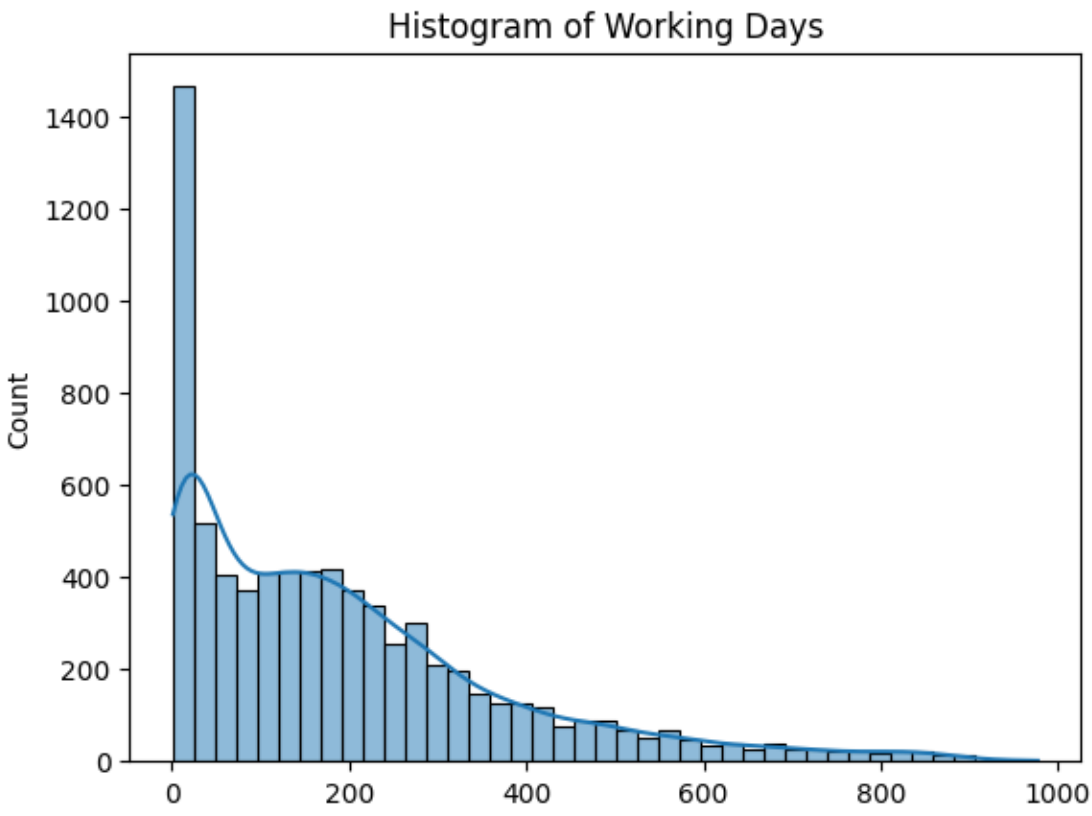
We will use the 2-Sample T-Test to test the hypothess defined above

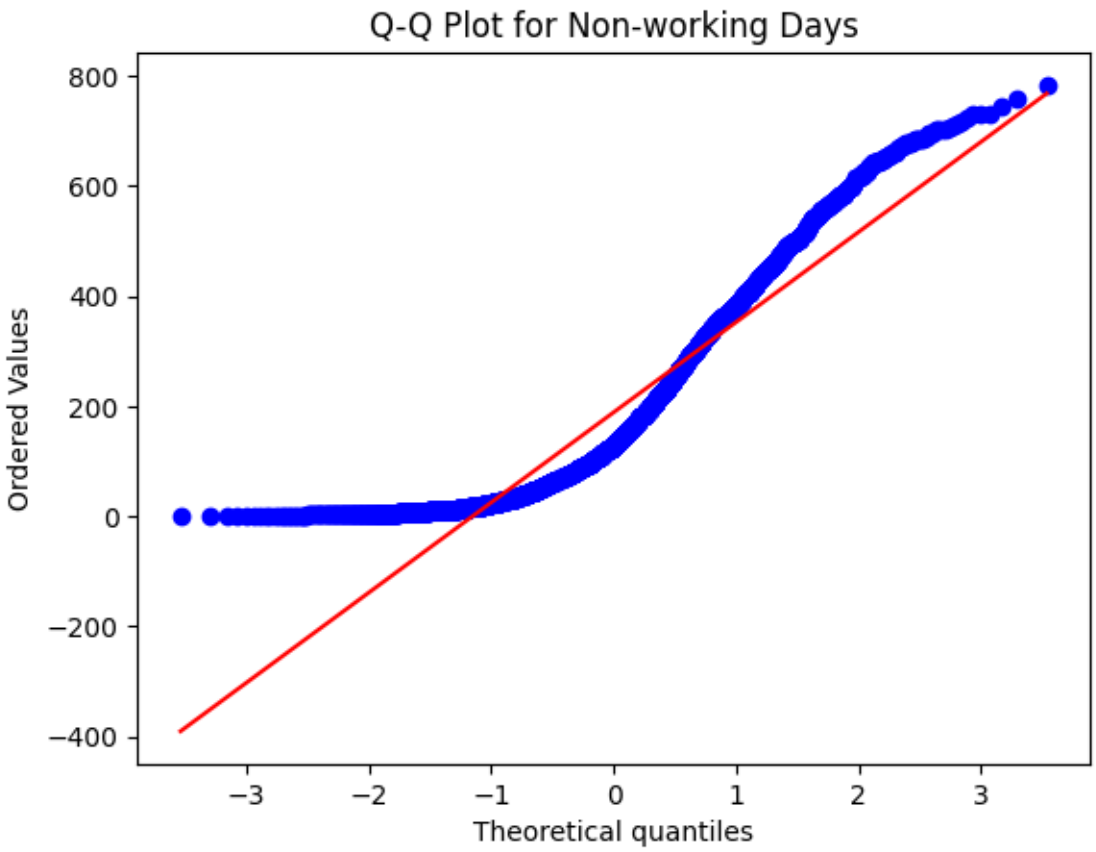
[73]:
```
# Group the data based on working day
data_group1 = df[df['workingday'] == 0]['count'].values  # Non-working days
data_group2 = df[df['workingday'] == 1]['count'].values  # Working days

# Shapiro-Wilk test for normality
shapiro_test_group1 = stats.shapiro(data_group1)
shapiro_test_group2 = stats.shapiro(data_group2)

print(f"Shapiro-Wilk test p-value for non-working days: {shapiro_test_group1.
 ↪pvalue}")
print(f"Shapiro-Wilk test p-value for working days: {shapiro_test_group2.
 ↪pvalue}")

# Check normality with histograms and Q-Q plots
sns.histplot(data_group1, kde=True)
plt.title('Histogram of Non-working Days')
plt.show()

sns.histplot(data_group2, kde=True)
plt.title('Histogram of Working Days')
plt.show()
```

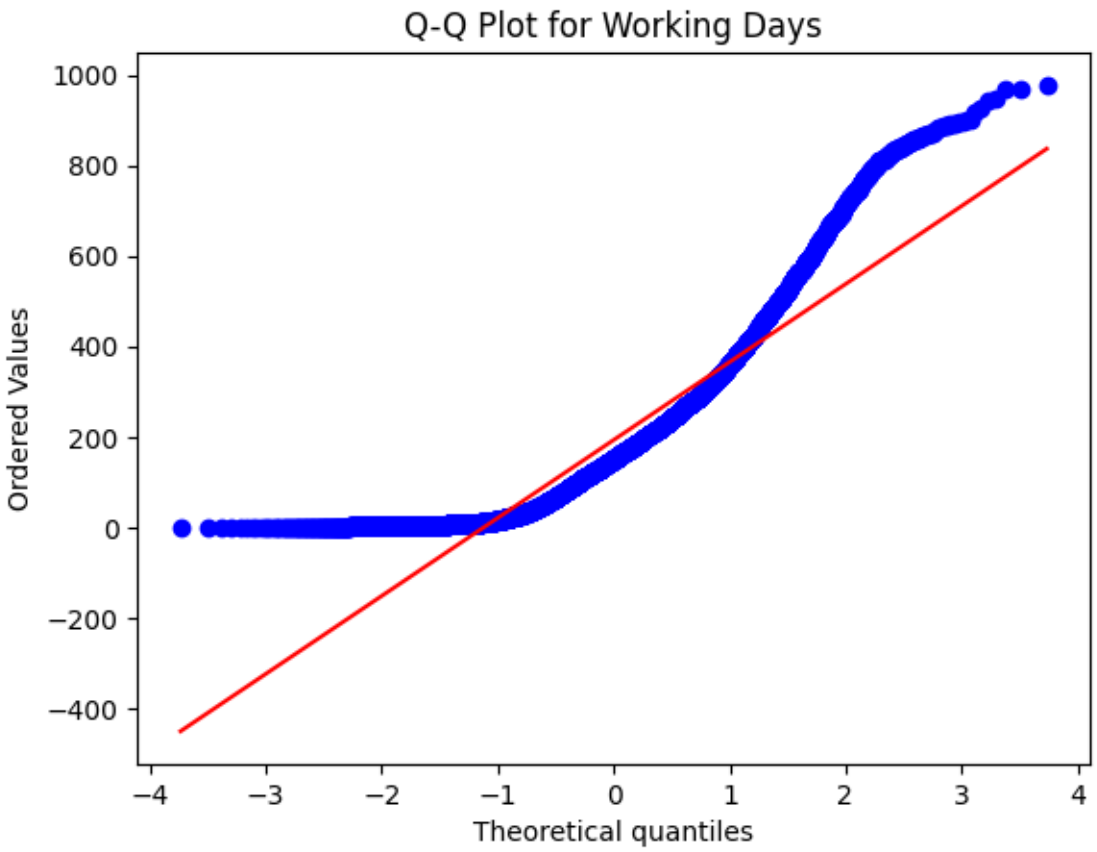
```
stats.probplot(data_group1, dist="norm", plot=plt)
plt.title('Q-Q Plot for Non-working Days')
plt.show()

stats.probplot(data_group2, dist="norm", plot=plt)
plt.title('Q-Q Plot for Working Days')
plt.show()

# Levene's test for equal variances
_, p_value_var = stats.levene(data_group1, data_group2)
print(f"Levene's test p-value: {p_value_var}")

# Perform the appropriate t-test
if p_value_var > 0.05:
    t_statistic, p_value = stats.ttest_ind(a=data_group1, b=data_group2,␣
 ↪equal_var=True)
else:
    t_statistic, p_value = stats.ttest_ind(a=data_group1, b=data_group2,␣
 ↪equal_var=False)

print(f"T-test statistic: {t_statistic}")
print(f"P-value: {p_value}")

# Significance level
alpha = 0.05

# Decision
if p_value <= alpha:
    print("\nReject the Null Hypothesis. Working Day affects the number of␣
 ↪cycles rented.")
else:
    print("Do not reject the Null Hypothesis. No evidence that Working Day␣
 ↪affects the number of cycles rented.")
```

```
/usr/local/lib/python3.10/dist-packages/scipy/stats/_axis_nan_policy.py:531:
UserWarning: scipy.stats.shapiro: For N > 5000, computed p-value may not be
accurate. Current N is 7412.
  res = hypotest_fun_out(*samples, **kwds)

Shapiro-Wilk test p-value for non-working days: 4.4728547627911074e-45
Shapiro-Wilk test p-value for working days: 2.2521124830019574e-61
```


Histogram of Non-working Days
Count
0
100
200
300
400
500
600
700
800


Histogram of Working Days
Count
0
200
400
600
800
1000
1200
1400
0
200
400
600
800
1000


Q-Q Plot for Non-working Days
Ordered Values
Theoretical quantiles
800
600
400
200
0
−200
−400
−3
−2
−1
0
1
2
3




```
Levene's test p-value: 0.9437823280916695
T-test statistic: -1.2096277376026694
P-value: 0.22644804226361348
Do not reject the Null Hypothesis. No evidence that Working Day affects the
number of cycles rented.
```

Using Shapiro-wilk test the rental counts are normally distributed for both working days and non-working days.

7 Hypothesis Testing - 3

Null Hypothesis: Number of cycles rented is similar in different weather

Alternate Hypothesis: Number of cycles rented is not similar in different weather

Significance level (alpha): 0.05

Here, we will use the ANOVA to test the hypothess defined above

[70]:
```
gp1 = df[df['weather']==1]['count'].values
gp2 = df[df['weather']==2]['count'].values
gp3 = df[df['weather']==3]['count'].values
```

```
gp4 = df[df['weather']==4]['count'].values

# conduct the one-way anova
stats.f_oneway(gp1, gp2, gp3, gp4)
```

[70]:
```
F_onewayResult(statistic=65.53024112793271, pvalue=5.482069475935669e-42)
```

Since p-value is less than 0.05, we reject the null hypothesis. This implies that Number of cycles rented is not similar in different weather conditions

8 Hypothesis Testing - 4

Null Hypothesis: Number of cycles rented is similar in different season

Alternate Hypothesis: Number of cycles rented is not similar in different season

Significance level (alpha): 0.05

Here, we will use the ANOVA to test the hypothess defined above

[71]:
```
gp5 = df[df['season']==1]['count'].values
gp6 = df[df['season']==2]['count'].values
gp7 = df[df['season']==3]['count'].values
gp8 = df[df['season']==4]['count'].values

# conduct the one-way anova
stats.f_oneway(gp5, gp6, gp7, gp8)
```

[71]:
```
F_onewayResult(statistic=236.94671081032106, pvalue=6.164843386499654e-149)
```

Since p-value is less than 0.05, we reject the null hypothesis. This implies that Number of cycles rented is not similar in different season conditions

Insights

In summer and fall seasons more bikes are rented as compared to other seasons. Whenever its a holiday more bikes are rented.

It is also clear from the workingday also that whenever day is holiday or weekend, slightly more bikes were rented.

Whenever there is rain, thunderstorm, snow or fog, there were less bikes were rented. Whenever the humidity is less than 20, number of bikes rented is very very low.

Whenever the temperature is less than 10, number of bikes rented is less.

Whenever the windspeed is greater than 35, number of bikes rented is less.

Recommendations

In summer and fall seasons the company should have more bikes in stock to be rented. Because the demand in these seasons is higher as compared to other seasons.

With a significance level of 0.05, workingday has no effect on the number of bikes being rented.

In very low humid days, company should have less bikes in the stock to be rented. Whenever temprature is less than 10 or in very cold days, company should have less bikes.

Whenever the windspeed is greater than 35 or in thunderstorms, company should have less bikes in stock to be rented.